

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [4]: #Read CSV file
df = pd.read_csv('Telco-Customer-Churn.csv')
df
```

Out[4]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	l
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	
...	...	...	...	...	...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	...	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	...	
7040	4801-JJAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	...	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	...	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	...	

7043 rows × 21 columns

```
In [3]: #Top 5 rows
df.head()
```

Out[3]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	l
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	...	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	...	
7040	4801-JJAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	...	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	...	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	...	

5 rows × 21 columns

```
In [4]: #Bottom 5 rows
df.tail()
```

Out[4]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	...	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	...	
7040	4801-JJAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	...	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	...	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	...	

5 rows × 21 columns

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines          7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection       7043 non-null   object
12  TechSupport            7043 non-null   object
13  StreamingTV            7043 non-null   object
14  StreamingMovies        7043 non-null   object
15  Contract               7043 non-null   object
16  PaperlessBilling       7043 non-null   object
17  PaymentMethod          7043 non-null   object
18  MonthlyCharges         7043 non-null   float64
19  TotalCharges           7043 non-null   object
20  Churn                  7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
In [7]: #For totalcharges replacing
df["TotalCharges"] = df ["TotalCharges"].replace(" ", "0")
df["TotalCharges"] = df ["TotalCharges"].astype("float")
```

```
In [8]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines          7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection       7043 non-null   object
12  TechSupport            7043 non-null   object
13  StreamingTV            7043 non-null   object
14  StreamingMovies        7043 non-null   object
15  Contract               7043 non-null   object
16  PaperlessBilling       7043 non-null   object
17  PaymentMethod          7043 non-null   object
18  MonthlyCharges         7043 non-null   float64
19  TotalCharges           7043 non-null   float64
20  Churn                  7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB
```

```
In [9]: df.isnull()
```

Out[9]:

Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	Streaming
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
...	...	...	...	...	...	...	...	...	...	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	
False	False	False	False	False	False	...	False	False	False	



```
In [10]: df.describe()
```

Out[10]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

```
In [11]: df.duplicated().sum()
```

```
Out[11]: 0
```

Converted 0 and 1 values of senior citizen to yes/no to make it easier to understand

```
In [12]: def conv(value):
          if value == 1:
              return "Yes"
          else:
              return "No"

          df['SeniorCitizen'] = df["SeniorCitizen"].apply(conv)
```

```
In [14]: df.head(20)
```

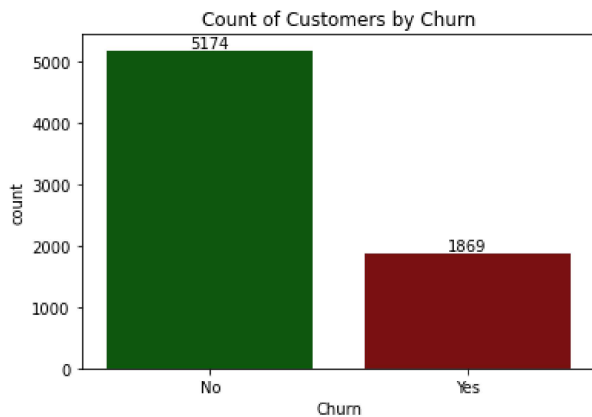
```
Out[14]:
```

dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	Streaming
No	1	No	No phone service	DSL	No	...	No	No	No	
No	34	Yes	No	DSL	Yes	...	Yes	No	No	
No	2	Yes	No	DSL	Yes	...	No	No	No	
No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	
No	2	Yes	No	Fiber optic	No	...	No	No	No	
No	8	Yes	Yes	Fiber optic	No	...	Yes	No	Yes	
Yes	22	Yes	Yes	Fiber optic	No	...	No	No	Yes	
No	10	No	No phone service	DSL	Yes	...	No	No	No	
No	28	Yes	Yes	Fiber optic	No	...	Yes	Yes	Yes	
Yes	62	Yes	No	DSL	Yes	...	No	No	No	
Yes	13	Yes	No	DSL	Yes	...	No	No	No	
No	16	Yes	No	No	No internet service	...	No internet service	No internet service	No internet service	No
No	58	Yes	Yes	Fiber optic	No	...	Yes	No	Yes	
No	49	Yes	Yes	Fiber optic	No	...	Yes	No	Yes	
No	25	Yes	No	Fiber optic	Yes	...	Yes	Yes	Yes	
Yes	69	Yes	Yes	Fiber optic	Yes	...	Yes	Yes	Yes	
No	52	Yes	No	No	No internet service	...	No internet service	No internet service	No internet service	No
Yes	71	Yes	Yes	Fiber optic	Yes	...	Yes	No	Yes	
Yes	10	Yes	No	DSL	No	...	Yes	Yes	No	
No	21	Yes	No	Fiber optic	No	...	Yes	No	No	

```
In [33]: ax = sns.countplot(
    x='Churn',
    data=df,
    palette=['#006400', '#8B0000'] # Dark Green, Dark Red
)

for p in ax.patches:
    ax.annotate(
        f'{int(p.get_height())}',
        (p.get_x() + p.get_width() / 2, p.get_height()),
        ha='center',
        va='bottom'
    )

plt.title("Count of Customers by Churn")
plt.show()
```

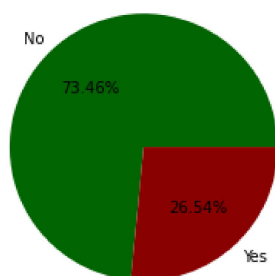


```
In [34]: gb = df.groupby('Churn').agg({'Churn': 'count'})

plt.pie(
    gb['Churn'],
    labels=gb.index,
    autopct='%1.2f%%',
    colors=['#006400', '#8B0000']
)

plt.title("Percentage of Churned Customers", fontsize=12)
plt.show()
```

Percentage of Churned Customers



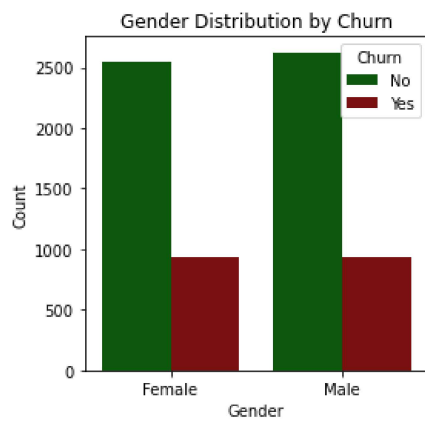
From the given pie chart we can conclude that 26.54% of our customers have churned out And explore the reason behind it

```
In [40]: plt.figure(figsize=(4,4))

sns.countplot(
    x="gender",
    data=df,
    hue="Churn",
    palette=['#006400', '#8B0000']
)

plt.title("Gender Distribution by Churn")
plt.xlabel("Gender")
plt.ylabel("Count")

plt.show()
```



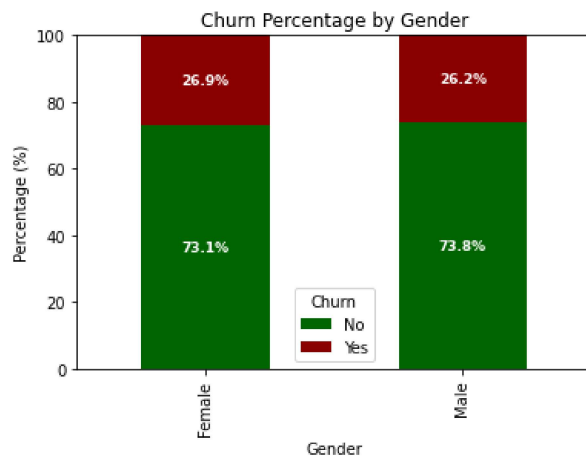
```
In [5]: # Create a cross-tab and convert counts to percentages
ct = pd.crosstab(df['gender'], df['Churn'], normalize='index') * 100

# Plot 100% stacked bar chart
ax = ct.plot(
    kind='bar',
    stacked=True,
    figsize=(6,4),
    color=['#006400', '#8B0000']
)

# Add percentage labels manually
for i in range(len(ct)):
    cumulative = 0
    for j in range(len(ct.columns)):
        value = ct.iloc[i, j]
        if value > 0:
            ax.text(
                i,
                cumulative + value/2,
                f'{value:.1f}%',
                ha='center',
                va='center',
                color='white',
                fontsize=9,
                fontweight='bold'
            )
            cumulative += value

plt.title("Churn Percentage by Gender")
plt.xlabel("Gender")
plt.ylabel("Percentage (%)")
plt.legend(title="Churn")
plt.ylim(0,100)

plt.show()
```

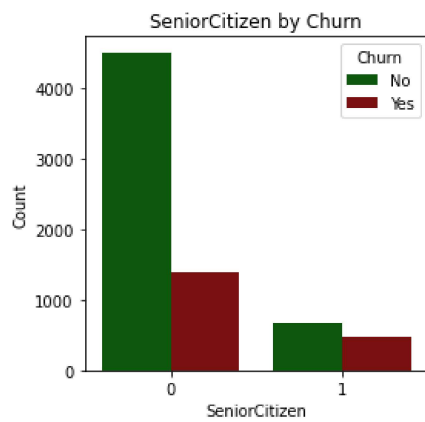


```
In [42]: plt.figure(figsize=(4,4))

sns.countplot(
    x="SeniorCitizen",
    data=df,
    hue="Churn",
    palette=['#006400', '#8B0000']
)

plt.title("SeniorCitizen by Churn")
plt.xlabel("SeniorCitizen")
plt.ylabel("Count")

plt.show()
```



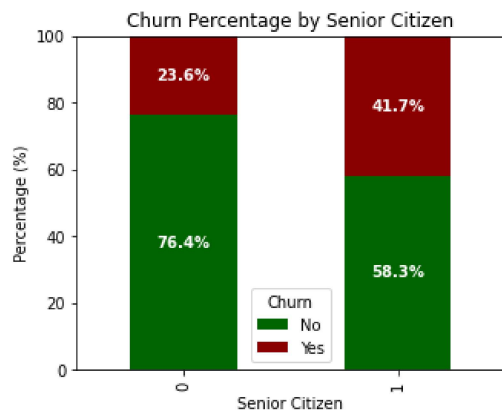

```
In [45]: # Create a cross-tab and convert counts to percentages
ct = pd.crosstab(df['SeniorCitizen'], df['Churn'], normalize='index') * 100

# Plot 100% stacked bar chart
ax = ct.plot(
    kind='bar',
    stacked=True,
    figsize=(5,4),
    color=['#006400', '#8B0000']
)

# Add percentage labels manually
for i in range(len(ct)):
    cumulative = 0
    for j in range(len(ct.columns)):
        value = ct.iloc[i, j]
        if value > 0:
            ax.text(
                i,
                cumulative + value / 2,
                f'{value:.1f}%',
                ha='center',
                va='center',
                color='white',
                fontsize=10,
                fontweight='bold'
            )
            cumulative += value

plt.title("Churn Percentage by Senior Citizen")
plt.xlabel("Senior Citizen")
plt.ylabel("Percentage (%)")
plt.legend(title="Churn")
plt.ylim(0, 100)

plt.show()
```



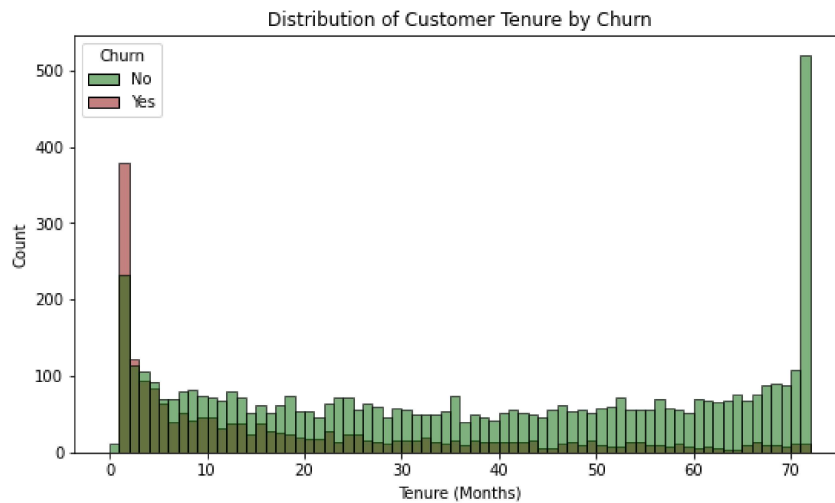
```
In [9]: plt.figure(figsize=(9,5))

sns.histplot(
    x="tenure",
    data=df,
    bins=72,
    hue="Churn",
    palette=['#006400', '#8B0000']
)

plt.title("Distribution of Customer Tenure by Churn")
plt.xlabel("Tenure (Months)")
plt.ylabel("Count")

plt.show()

#People who have used our service for a long time have stayed and people who have used our service
#1 or 2 Moths have churned
```



```
In [12]: plt.figure(figsize=(5,4))

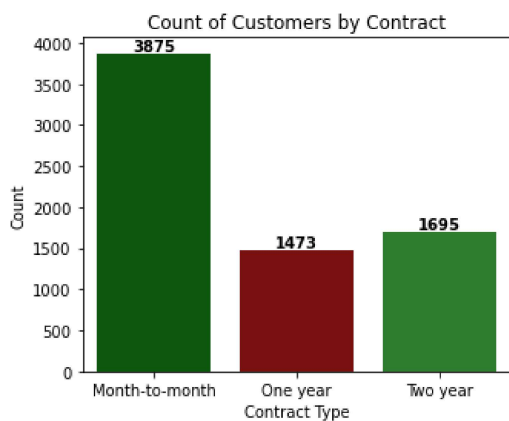
ax = sns.countplot(
    x="Contract",
    data=df,
    palette=['#006400', '#8B0000', '#228B22']
)

for p in ax.patches:
    ax.annotate(
        str(int(p.get_height())),
        (p.get_x() + p.get_width()/2., p.get_height()),
        ha='center',
        va='bottom',
        fontsize=10,
        fontweight='bold'
    )

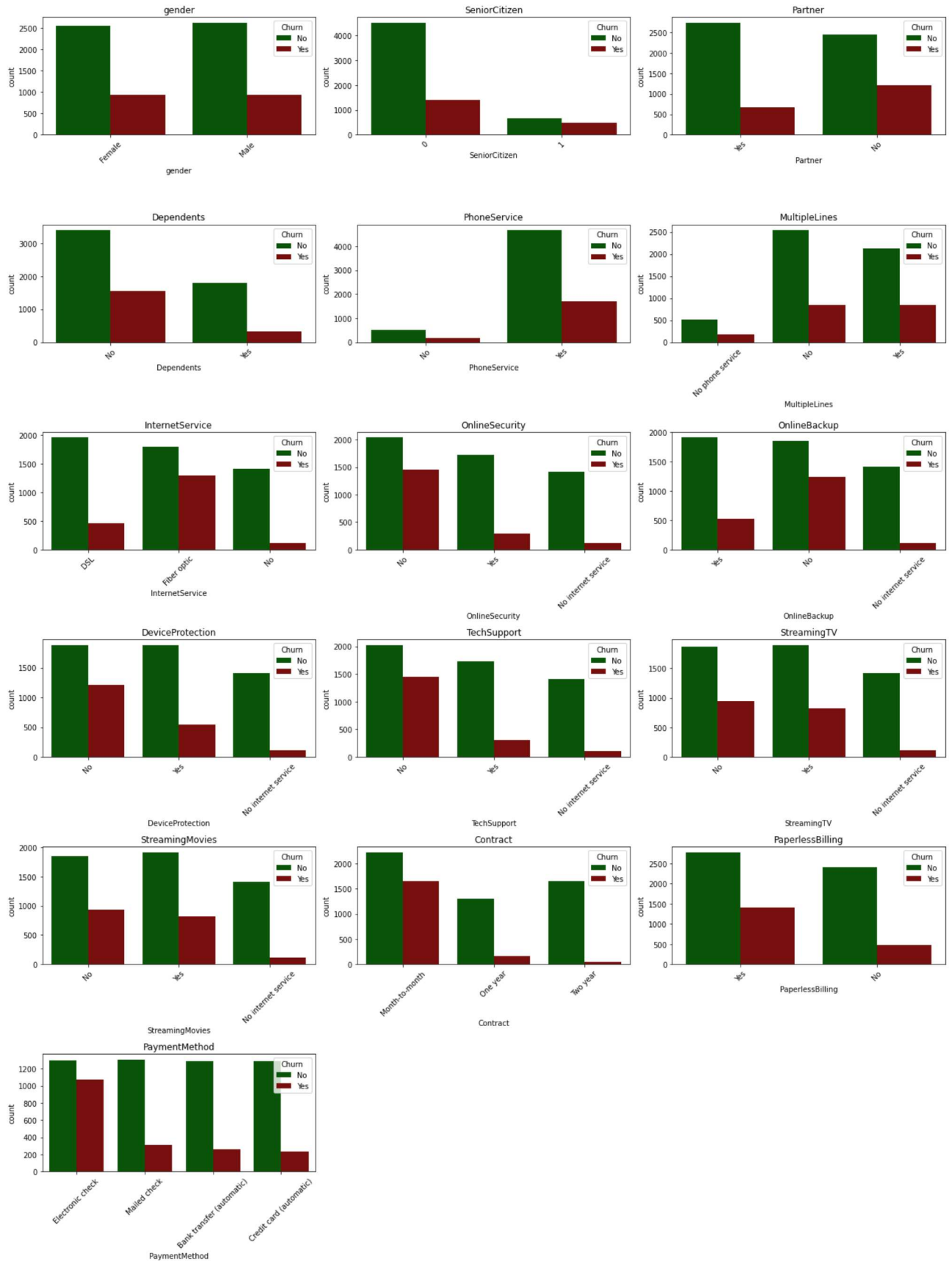
plt.title("Count of Customers by Contract")
plt.xlabel("Contract Type")
plt.ylabel("Count")

plt.show()

#People who have month to month contract are likely to churn then from those who have 1 or 2 years or contract
```



```
In [17]: cat_cols = [  
    'gender', 'SeniorCitizen', 'Partner', 'Dependents',  
    'PhoneService', 'MultipleLines', 'InternetService',  
    'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',  
    'TechSupport', 'StreamingTV', 'StreamingMovies',  
    'Contract', 'PaperlessBilling', 'PaymentMethod'  
]  
  
fig, axes = plt.subplots(6, 3, figsize=(18, 24))  
axes = axes.flatten()  
  
for i, col in enumerate(cat_cols):  
    sns.countplot(  
        x=col,  
        data=df,  
        hue='Churn',  
        palette=['#006400', '#8B0000'],  
        ax=axes[i]  
    )  
  
    axes[i].set_title(col)  
    axes[i].tick_params(axis='x', rotation=45)  
  
for j in range(len(cat_cols), len(axes)):  
    fig.delaxes(axes[j])  
  
plt.tight_layout()  
plt.show()
```



Analysis of Categorical Features

The following count plots illustrate the distribution of customers across various categorical variables, with churn status represented by different colors. This analysis helps identify potential relationships between customer attributes, service usage, contract types, payment methods, and customer churn.

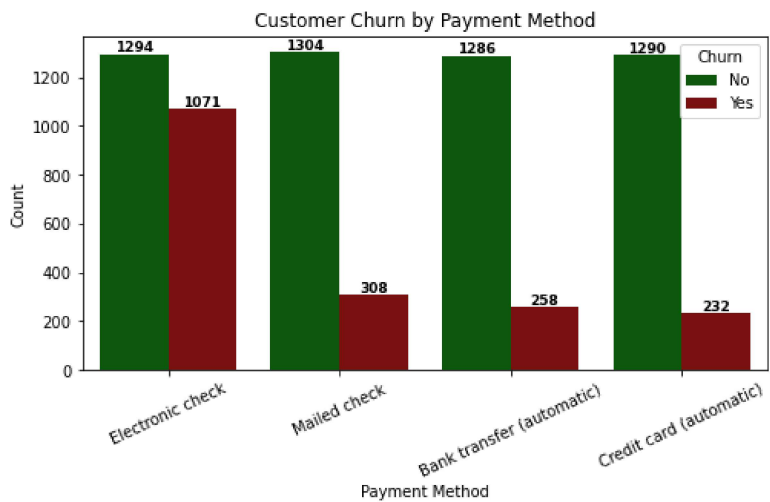
```
In [18]: plt.figure(figsize=(8,4))

ax = sns.countplot(
    x="PaymentMethod",
    data=df,
    hue="Churn",
    palette=['#006400', '#8B0000']
)

# Add count Labels
for p in ax.patches:
    ax.annotate(
        str(int(p.get_height())),
        (p.get_x() + p.get_width()/2., p.get_height()),
        ha='center',
        va='bottom',
        fontsize=9,
        fontweight='bold'
    )

plt.title("Customer Churn by Payment Method")
plt.xlabel("Payment Method")
plt.ylabel("Count")
plt.xticks(rotation=25)

plt.show()
```



Customer is likely to churn when he is using electronic check as a payment method

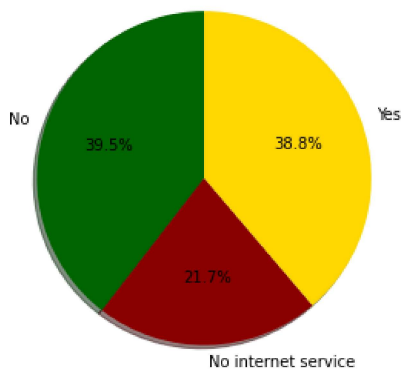
```
In [19]: gb = df.groupby('StreamingMovies').agg({'customerID': 'count'})

plt.figure(figsize=(5,5))

plt.pie(
    gb['customerID'],
    labels=gb.index,
    autopct='%1.1f%%',
    colors=['#006400', '#8B0000', '#FFD700'],
    startangle=90,
    shadow=True
)

plt.title("Customer Distribution by Streaming Movies")
plt.show()
```

Customer Distribution by Streaming Movies



Count customers by StreamingMovies

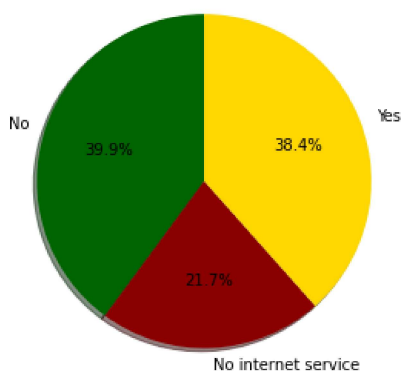
```
In [20]: gb = df.groupby('StreamingTV').agg({'customerID': 'count'})

plt.figure(figsize=(5,5))

plt.pie(
    gb['customerID'],
    labels=gb.index,
    autopct='%1.1f%%',
    colors=['#006400', '#8B0000', '#FFD700'],
    startangle=90,
    shadow=True
)

plt.title("Customer Distribution by Streaming TV")
plt.show()
```

Customer Distribution by Streaming TV



Count customers by StreamingTV

In []: